

**École polytechnique de Louvain**

# **Tricolor:**

Computational Analysis and AI Exploration of an  
Early Hexagonal Board Game

Author: **Andrei BOURCEANU**  
Supervisors: **Eric PIETTE, Lisa ROUGETET**  
Readers: **Hélène VERHAEGHE, Benoît RONVAL**  
Academic year 2025–2026  
Master [120] in Computer Science

*I [...] like to give the maximum in everything I do. The maximum I have. The maximum I can give. I am not perfect. But if I do something, I do it [as best I can].*

Reinhold Messner

# CONTENTS

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>Acknowledgments</b>                                              | <b>v</b>  |
| <b>1 Historical Background and Motivation</b>                       | <b>1</b>  |
| 1.1 Tricolor and its Inventor . . . . .                             | 1         |
| 1.2 Motivation . . . . .                                            | 2         |
| 1.3 Game Mechanics . . . . .                                        | 2         |
| 1.4 Similar previous Games . . . . .                                | 3         |
| 1.5 What was studied so far . . . . .                               | 3         |
| <b>2 Game Rules</b>                                                 | <b>7</b>  |
| 2.1 Number of players . . . . .                                     | 7         |
| 2.2 Board and Pieces setup . . . . .                                | 7         |
| 2.3 Gameplay and Rules . . . . .                                    | 8         |
| 2.3.1 How movement works . . . . .                                  | 9         |
| 2.3.2 How the destination tile influences pieces movement . . . . . | 10        |
| 2.4 Game ending and winner of the game . . . . .                    | 12        |
| 2.4.1 Additional rule . . . . .                                     | 12        |
| <b>3 Implementation of Tricolor</b>                                 | <b>15</b> |
| 3.1 Game Logic . . . . .                                            | 15        |
| 3.1.1 Board State. . . . .                                          | 15        |
| 3.1.2 Actions. . . . .                                              | 17        |
| 3.1.3 Game. . . . .                                                 | 18        |
| 3.2 Basic Agents and User Interface . . . . .                       | 18        |
| 3.2.1 Random Agent . . . . .                                        | 18        |
| 3.2.2 Human Agent and User Interface. . . . .                       | 18        |
| <b>4 Game Metrics</b>                                               | <b>21</b> |
| 4.1 Duration . . . . .                                              | 21        |
| 4.1.1 Metric . . . . .                                              | 21        |
| 4.1.2 Data Interpretation. . . . .                                  | 21        |
| 4.2 Branching Factor. . . . .                                       | 22        |
| 4.2.1 Metric . . . . .                                              | 22        |
| 4.2.2 Data Interpretation. . . . .                                  | 22        |
| 4.3 Game Complexity . . . . .                                       | 23        |
| 4.3.1 Game Tree Complexity. . . . .                                 | 23        |
| 4.3.2 State Tree Complexity . . . . .                               | 24        |
| 4.4 Game Outcome. . . . .                                           | 25        |
| 4.4.1 Metric . . . . .                                              | 25        |
| 4.4.2 Data Interpretation. . . . .                                  | 25        |

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| 4.5      | State Evaluation . . . . .                          | 26        |
| 4.5.1    | Narrowness . . . . .                                | 26        |
| 4.5.2    | Decisiveness . . . . .                              | 27        |
| 4.5.3    | Stability . . . . .                                 | 27        |
| 4.5.4    | Drama . . . . .                                     | 28        |
| 4.5.5    | Game Evaluations Examples . . . . .                 | 29        |
| <b>5</b> | <b>Game Analysis for Advanced AI Agents</b>         | <b>33</b> |
| 5.1      | Game Complexity . . . . .                           | 33        |
| 5.2      | Heuristics . . . . .                                | 33        |
| 5.2.1    | First insight . . . . .                             | 33        |
| 5.2.2    | Second insight . . . . .                            | 34        |
| 5.2.3    | Third insight . . . . .                             | 34        |
| 5.2.4    | Combining the heuristics. . . . .                   | 36        |
| 5.3      | Alpha-Beta Agent . . . . .                          | 37        |
| 5.4      | Monte Carlo Tree Search Agent . . . . .             | 37        |
| 5.5      | Expert Iteration . . . . .                          | 38        |
| <b>6</b> | <b>Performance Analysis of the Agents</b>           | <b>39</b> |
| 6.1      | Random . . . . .                                    | 39        |
| 6.2      | Alpha-Beta vs MCTS . . . . .                        | 40        |
| 6.3      | Game Metrics using AI Agents . . . . .              | 40        |
| 6.3.1    | Duration . . . . .                                  | 41        |
| 6.3.2    | Branching Factor. . . . .                           | 41        |
| 6.3.3    | Decisiveness . . . . .                              | 41        |
| 6.3.4    | Stability . . . . .                                 | 42        |
| <b>A</b> | <b>Game metrics plots</b>                           | <b>45</b> |
| A.1      | Random Agent vs Random Agent simulations . . . . .  | 46        |
| A.2      | Alpha-Beta Agent vs MCTS Agent simulations. . . . . | 49        |
|          | <b>Bibliography</b>                                 | <b>53</b> |

## ACKNOWLEDGMENTS

I would like to thank Lisa Rougetet (*Université de Bretagne Occidentale*) for her collaboration on this work. She kindly provided historical materials related to the Tricolor game, including photographs of the original patent, as well as additional resources from historical books. These contributions were essential to the historical component of this study.



## 1

# HISTORICAL BACKGROUND AND MOTIVATION

## 1.1 TRICOLOR AND ITS INVENTOR

**Tricolor** is a board game invented in 1930 by **Maurice Kraitchik** (1882 – 1957) who was a Belgian mathematician, initially born in Russia, whose main interests were number theory and recreational mathematics [1]. He holds the patent for the game, which contains a photo of him (Figure 1.1), an official certificate (Figure 1.2) and the rules of Tricolor written in French.



Figure 1.1: Photo of Maurice Kraitchik from his patent for Tricolor

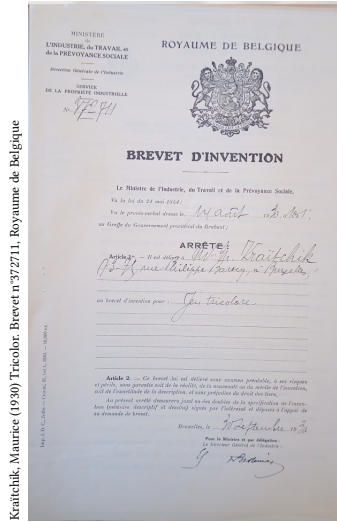


Figure 1.2: Photo of the official patent certificate for the invention of Tricolor

## 1.2 MOTIVATION

**Tricolor** is the first board game in history to use a hexagonal tiling 1.3. The unique gameplay features this structure enables, as well as its influence on later games adopting hexagonal tilings, constitute the main motivation for studying Tricolor.

The main objectives of this study on **Tricolor** are as follows:

1. To present the historical context of the game, including prior developments and subsequent research.
2. To analyze the game's features and metrics, as well as its computational and strategic complexity.
3. To develop AI agents capable of playing the game at a high level.

## 1.3 GAME MECHANICS

Tricolor is a complex combinatorial game due to the high number of possible moves at each turn. The main features that make the game unique are the hexagonal board tiling (such a tiling looks like the one in Figure 1.3), the ability to stack pieces on top of each other, winning by capturing the opponent's pieces and the need to control the board in order gain an advantage. So far, Tricolor is known by historians as the first game in history to be using a hexagonal board tiling, which even today is quite a rare tiling used in games.

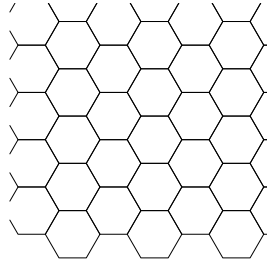


Figure 1.3: Example of general hex tiling design

## 1.4 SIMILAR PREVIOUS GAMES

The only game known by historians to be similar to Tricolor and invented before is **Laska** (also known as **Lasca**). Laska is a board game invented by the chess world champion and German mathematician Emanuel Lasker (1868–1941) in 1911 [2]. The game has similar mechanics to Tricolor, such as being a combinatorial game, being able to stack pieces on top of each other by moving them around and controlling the board by blocking the opponent and capturing his pieces (see Figure 1.4).

AI techniques such as Monte Carlo Tree Search and Deep Neural Networks have already been investigated for playing Lasca, but they have been shown to perform poorly. This is partly due to the fact that Lasca allows pieces to be stacked on top of each other<sup>1</sup>, which increases the number of channels required in tensor-based game representations. Consequently, this also leads to a larger number of trainable parameters in the neural network architecture [3].

As Maurice Kraitichik claims in the patent description, Tricolor was directly inspired from Laska.



Figure 1.4: Image with Laska board

## 1.5 WHAT WAS STUDIED SO FAR

To date, there are no known papers or articles—whether physical or online—addressing the strategies or mathematical properties of Tricolor, apart from the book *La mathématique des*

<sup>1</sup>as in Tricolor

## 1

*jeux ou récréations mathématiques* by Maurice Kraitchik (Stevens Frères, 1930). This work contains a wide range of mathematical problems of varying difficulty, as well as a chapter dedicated to Tricolor. In this chapter, the author introduces the rules of the game, defines a notation for possible moves, and presents two example games (see Figure 1.5). He also discusses several endgame scenarios in which one player holds a slight material advantage and illustrates how the advantaged player can secure a win in the shortest possible way. While this analysis provides useful insight into certain strategic elements, it only covers a limited subset of the game's possible positions. As a result, a more comprehensive study is still required to develop robust strategic theories for the game.

| N° | Blancs  | Noirs   | N° | Blancs        | Noirs            | N° | Blancs           | Noirs            |
|----|---------|---------|----|---------------|------------------|----|------------------|------------------|
| 1  | 6 — 12  | 51 — 44 | 12 | 17 — 18       | 56 — 50          | 23 | $31^2 \times 48$ | $50 \times 48$   |
| 2  | 13 — 12 | 45 — 44 | 13 | 12 — 38       | 44 — 21          | 24 | $31 — 33$        | $48^2 \times 33$ |
| 3  | 1 — 7   | 57 — 52 | 14 | $38^1 — 30$   | $21^1 : 30$      | 25 | $18^2 : 33$      | $48^2 \times 33$ |
| 4  | 2 — 7   | 58 — 52 | 15 | $15^2 : 30$   | $47^2 : 30$      | 26 | 18 — 11          | 33 — 24          |
| 5  | 8 — 3   | 60 — 59 | 16 | $3 \times 21$ | $59 \times 38$   | 27 | 11 — 10          | $24^1 — 17$      |
| 6  | 9 — 3   | 53 — 59 | 17 | $21 : 30$     | $38 : 30$        | 28 | 10 : 17          | 24 : 17          |
| 7  | 4 — 10  | 46 — 47 | 18 | 15 — 9        | $30^3 : 9$       |    | —                | gagne            |
| 8  | 5 — 10  | 54 — 47 | 19 | 7 : 9         | $30 — 39$        |    |                  |                  |
| 9  | 14 — 15 | 61 — 55 | 20 | 9 — 10        | $39 — 52$        |    |                  |                  |
| 10 | 16 — 15 | 48 — 55 | 21 | 10 — 31       | $55^1 — 48$      |    |                  |                  |
| 11 | 11 — 18 | 49 — 50 | 22 | $31^2 : 48$   | $55^2 \times 48$ |    |                  |                  |

— 475 —

| N° | Blancs      | Noirs            | Rouges      | N° | Blancs         | Noirs            | Rouges    |
|----|-------------|------------------|-------------|----|----------------|------------------|-----------|
| 1  | 6 — 7       | 27 — 28          | 43 — 50     | 17 | 50 — 33        | 45 — 37          | 54 — 53   |
| 2  | 1 — 7       | 19 — 28          | 42 — 50     | 18 | $33 — 31$      | $37 — 29$        | $53 — 38$ |
| 3  | 2 — 3       | 45 — 52          | 56 — 55     | 19 | $31^3 : 29$    | $44^3 \times 29$ | $38 — 53$ |
| 4  | 8 — 3       | 51 — 52          | 49 — 55     | 20 | $31 \times 29$ | $44 — 36$        | $53 — 59$ |
| 5  | 4 — 10      | 36 — 44          | 26 — 34     | 21 | $29^1 — 20$    | $36 : 20$        | $59 — 60$ |
| 6  | 11 — 10     | 37 — 44          | 35 — 34     | 22 | $29 : 20$      |                  | $60 — 55$ |
| 7  | 9 — 10      | 58 — 52          | 60 — 61     | 23 | $20^3 — 21$    |                  | $55 — 48$ |
| 8  | 5 — 10      | 57 — 52          | 34 — 55     | 24 | 21 — 47        |                  | $48 — 41$ |
| 9  | 10 — 34     | 52 — 28          | 50 — 47     | 25 | $47^1 — 39$    |                  | $41 : 39$ |
| 10 | $34 — 31$   | $44^1 — 45$      | $47^2 : 45$ | 26 | $47 \times 39$ |                  |           |
| 11 | $31^1 — 40$ | $28^2 \times 45$ | $47 : 40$   |    | +              |                  |           |
| 12 | $31^2 : 40$ | $28^3 \times 31$ | $55^3 : 40$ |    |                |                  |           |
| 13 | 7 — 10      | 31 : 40          | 55 : 40     |    |                |                  |           |
| 14 | 3 — 24      | $45^1 — 37$      | $40 : 37$   |    |                |                  |           |
| 15 | 24 — 50     | 28 : 37          | 61 — 55     |    |                |                  |           |
| 16 | 10 — 31     | 37 — 44          | 55 — 54     |    |                |                  |           |

Figure 1.5: Example of 2 possible games to be played



# 2

## GAME RULES

Understanding a game's rules is the first step toward analyzing it. They provide the foundation for studying the game's complexity, developing strategies, and designing AI agents capable of playing at a high level.

In this chapter, we present the rules of **Tricolor** as described in the original patent, with one exception. In theory, a game of Tricolor can continue indefinitely if the players choose to do so, since the original rules do not include a condition that forces the game to end. This absence is problematic when analyzing the game through thousands of simulations,<sup>1</sup> as it may lead to excessively long games. To address this issue, we introduce an additional rule, presented at the end of this chapter, that guarantees termination.

### 2.1 NUMBER OF PLAYERS

In the original patent for Tricolor it's specified that the number of players can be 2 or 3, but in this paper I will only consider the 2 players version.

### 2.2 BOARD AND PIECES SETUP

The board uses a hexagonal tiling containing 3 different colors. Each player starts with 18 pieces on his side of the board as shown in Figure 2.1. Player 1 has the white pieces and player 2 the black pieces.

---

<sup>1</sup>Some games could otherwise last for an extremely large number of turns.

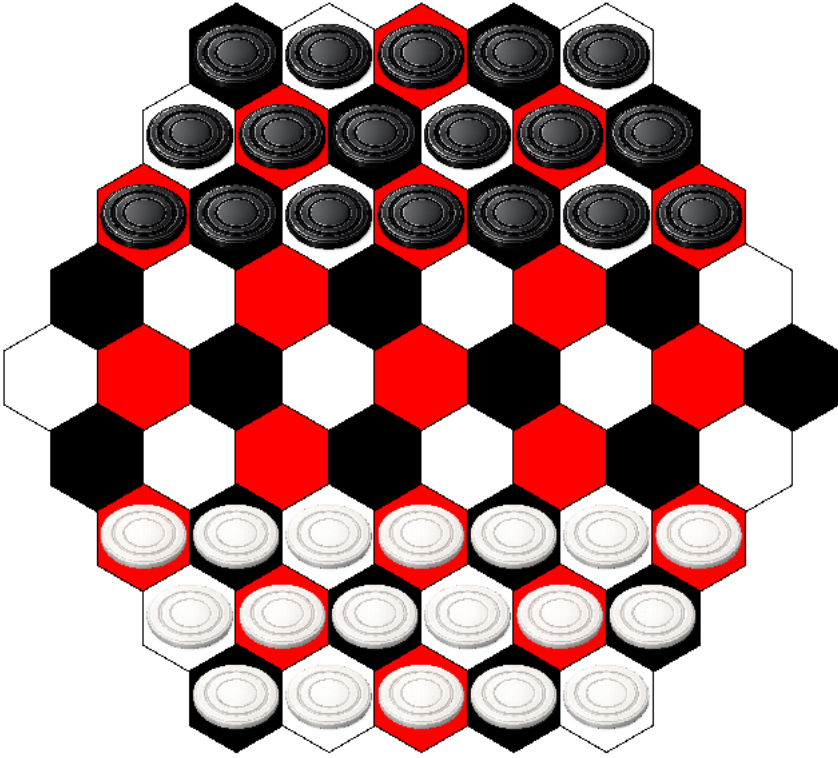


Figure 2.1: Initial board setup

## 2.3 GAMEPLAY AND RULES

Each player takes his turn to make one move. By convention, player 1 is always first to move when the game starts. A move consists in selecting one **stack** of pieces **controlled** by the player making the move and **move** any number of pieces<sup>2</sup> to an **available** tile.

### Definitions:

- A **stack** of pieces is a literal stack that can contain multiple pieces, both white and black. For example, at the start of the game, each player has 18 stacks containing only 1 friendly piece.
- A stack is said to be **controlled** by player 1 (resp. player 2) if the color of the piece on top is white (resp. black e.g. Figure 2.2).
- **Moving**  $x$  pieces from a tile to another means removing the first  $x$  pieces from the initial tile, from top to bottom, and placing them on the destination tile.

<sup>2</sup>at least 1

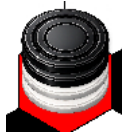


Figure 2.2: Example of a stack controlled by black

2

- When moving a stack of pieces, that stack has a so called **power**. The power of a stack isn't influenced by the number of the opponent's pieces. Therefore, the **power** of a stack containing  $x$  friendly pieces is the following:

$$\min(3, x) * color$$

, where *color* is a value depending on the color of the initial tile and is as follows:

- **white** = 1
- **black** = 2
- **red** = 3

As an example, consider Figure 2.3 and Figure 2.4. Note that the concept of **power** is also applied to the *defending* stack, which is the stack of pieces controlled by the opponent on the destination tile. The formula is the same as for the *attacking power* of a stack and it takes into account all the opponent pieces found on that tile.

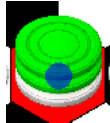


Figure 2.3: Example of an attacking stack (white) with power = 2 (from 2 selected pieces) \* 3 (color red) = 6



Figure 2.4: Example of an attacking stack with power = 3 (from 3 selected friendly pieces) \* 1 (color white) = 3

### 2.3.1 HOW MOVEMENT WORKS

The number of friendly pieces the current player decides to move defines how many tiles he can travel, but it's at most 3<sup>3</sup>. Therefore, the number of selected pieces belonging to

<sup>3</sup>e.g. if the player moves 7 friendly pieces the maximal distance he can travel is 3

the opponent doesn't influence the distance allowed to travel. When moving pieces from a tile to another, all the tiles in between must be empty <sup>4</sup>.

A move can be made in the directions shown by the Figure 2.5:

2

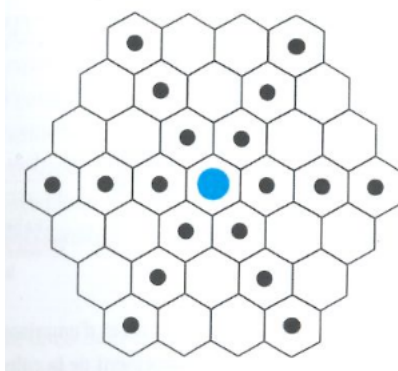


Figure 2.5: All six directions a stack can move in.

### 2.3.2 HOW THE DESTINATION TILE INFLUENCES PIECES MOVEMENT

When moving  $x$  pieces from a tile to another, there are 3 possible scenarios:

1. The destination tile is empty: in this case, there are no restrictions or exceptions regarding the move.
2. The destination tile is controlled by you: in this case, as in case 1, there are no restrictions or exceptions regarding the move. The friendly pieces you moved will be stacked on top of the friendly pieces on the destination tile. The opponent pieces you moved will be stacked on top of the opponent pieces on the destination tile, but **under** your pieces <sup>5</sup>. As an example, consider Figure 2.6 and Figure 2.7



Figure 2.6: Initial position

3. The destination tile is controlled by your opponent: in this case, there are again 3 scenarios:
  - (a) The **power** of the attacking stack is strictly higher than the **double** of the power of the defending stack: in this case, all the opponent's pieces on the

<sup>4</sup>If not, you can't make that move

<sup>5</sup>You can see this as a reordering of the pieces on the destination tile where all your pieces are on top and the opponent's at the bottom



Figure 2.7: Position after moving to friendly tile (all white pieces are stacked on top and all black pieces are stacked on the bottom)

2

destination tile are completely removed from the game. This move is called a **capture**. Note, however, that the opponent's pieces belonging to the attacker's stack still remain in the game. As an example, consider Figure 2.8 and Figure 2.9.



Figure 2.8: Before capture, the attacker (white) has power  $2 * 2 = 4$  and the defender (black) has power  $1 * 1 = 1$ , so it's a capture



Figure 2.9: After capturing, the black piece is removed

- (b) The **power** of the attacking stack is strictly higher than the power of the defending stack, but not sufficiently high to be a **capture**: in this case, the opponent's pieces on the destination tile are not removed from the game, but the attacking player gains the control of the defending stack. This move is called an **imprisonment**. As an example, consider Figure 2.10 and Figure 2.11.
- (c) The **power** of the attacking stack is lower than or equal to the power of the defending stack: in this case, the move isn't allowed.



Figure 2.10: Before imprisonment, the attacker (black) has power  $3 * 1 = 3$  and the defender (white) has power  $1 * 2 = 2$ , so it's an imprisonment



Figure 2.11: After imprisonment, black now controls the defender's tile

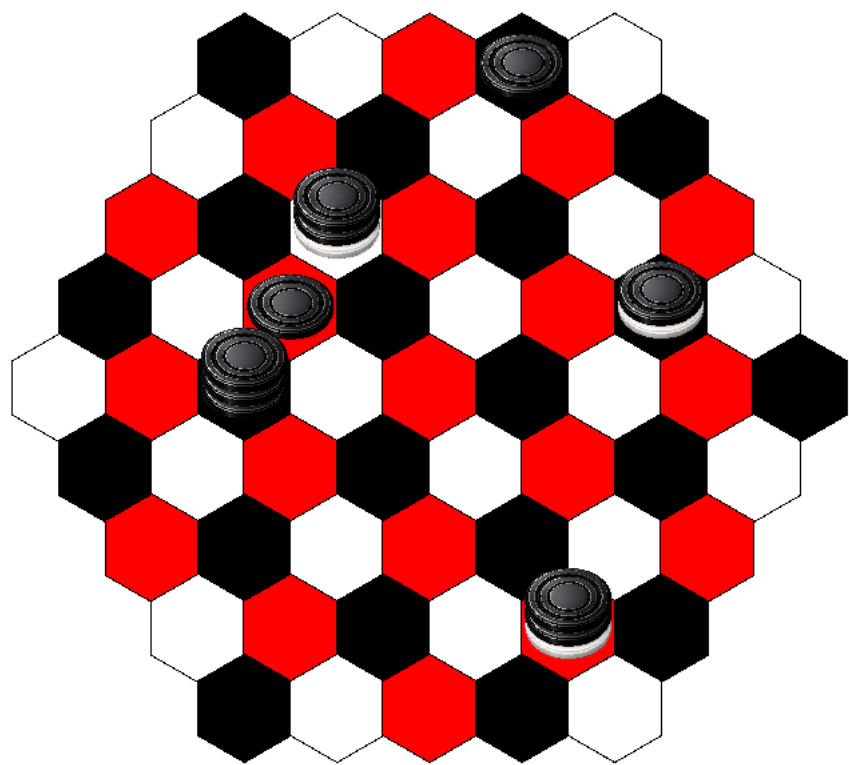
Note: when a "scenario 3" type of move (i.e. the destination tile is controlled by your opponent) is possible to a player, they are forced to make it. If multiple "scenario 3" moves are possible, the player can choose the one they want to make.

## 2.4 GAME ENDING AND WINNER OF THE GAME

A game ends when any player can't make any move. If this happens to a player, the other player wins the game. As an example, consider Figure 2.12

### 2.4.1 ADDITIONAL RULE

As explained in the introduction of this chapter, in the patent for the game there are no rules regarding how long a game can last. Therefore, because it's theoretically possible to get to positions where the best strategy for both players is to avoid the opponent's pieces, it's important to add a rule specifying that if no stack was captured in the last 100 turns (each player must play 50 moves), then the game ends and results in a draw. This would avoid infinite length games.



Black won

Figure 2.12: Example of a finished game won by black. White has no moves.



## 3

## 3

## IMPLEMENTATION OF TRICOLOR

The framework for the game **Tricolor** was implemented in C++. This choice was made because C++ is a low-level language that allows efficient memory management and runtime optimization.

### 3.1 GAME LOGIC

The game logic refers to the set of low-level structures that must be implemented to model the game Tricolor accurately.

These three structures are:

- **Board State**
- **Actions**
- **Game**

#### 3.1.1 BOARD STATE

The board state is one of the most important features of the game logic. This is because it must encode all the necessary information required to define a specific state of the board, but also because it needs to be easy to *copy*. For a proper AI analysis of games, being able to make fast copies of any board state is crucial as all modern AI algorithms (e.g. AlphaBeta and MCTS) need to do it hundreds or thousands of times [4].

In the search of the most compact board state representation, it's important to have the following insights:

- The board has 61 tiles, as you can see in Figure 3.1. Therefore, an array of size 61 can be used, where each element represents one single tile.
- Each player starts with 18 pieces (so there are 18 white pieces and 18 black pieces on the board). The number of pieces can't increase, because there is no such feature in the game. However, pieces of the same or different color can be stack on the same tile, which means, in theory, a tile could have 18 white pieces and 18 black pieces.

- Moreover, when a tile contains mixed colors pieces, it's impossible to deduce what player owns the tile, so it's required to have metadata describing the owner.
- It's important to know what player has to make the next move for the current pieces positions on the board.

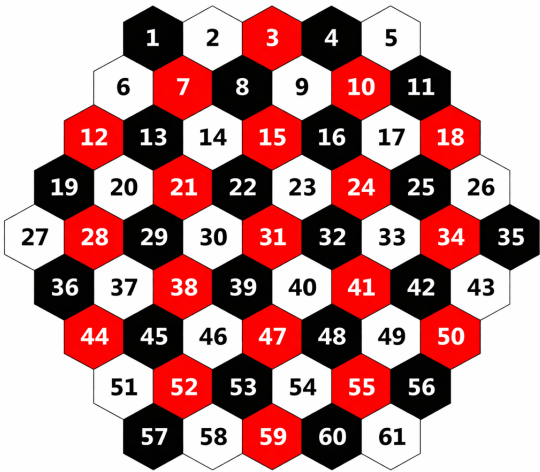


Figure 3.1: Board with numbered tiles from 1 to 61

With these 3 insights we can conclude that a good data structure for the board state is an array of length 61 (for the number of tiles of the board) where each element is a 16 bits number. This is because, for each tile, we need to maintain the number of white and black pieces (from 0 to 18 for each color, which means we need 5 bits for each color, so 10 bits in total), as well as the owner of the tile (there are only 2 players, so we only need 1 bit; notice that for the neutral tiles, i.e. the empty tiles, it's possible to deduce the owner of the tile by the fact that there are no pieces at all, so no need for a "third" owner). A visual representation of the bit masking of each tile can be seen in Figure 3.2. Finally, the player who has to move in the current position is maintained using a boolean value (0 = white has to move and 1 = black has to move).

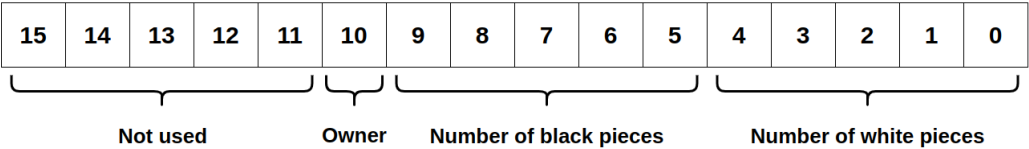


Figure 3.2: 16-bit integer visual representation masking the state of a board

This final compact structure of a board state has only a size of:

$$61 * 16 \text{ bits} + 8 \text{ bits} = 61 * 2 \text{ bytes} + 1 \text{ byte} = 123 \text{ bytes} \quad (3.1)$$

, which is really small. Therefore, making a copy of such a state is very fast.

Note: the different operations (getting the number of white pieces, black pieces, owner of a tile etc.) are done only using bitwise operations, making the implementation even faster than a naive one.

### 3.1.2 ACTIONS

In any game, actions are the bridge between the different board states. In any position, the player that has to make the next move needs a set of actions to choose from (e.g. Player 1 decides to move 3 pieces of this stack to 2 tiles to the left like in Figure 3.3). When talking about performance, a model has to:

1. generate all the possible actions for a board state as fast as possible
2. use the least memory possible for the **action** structure

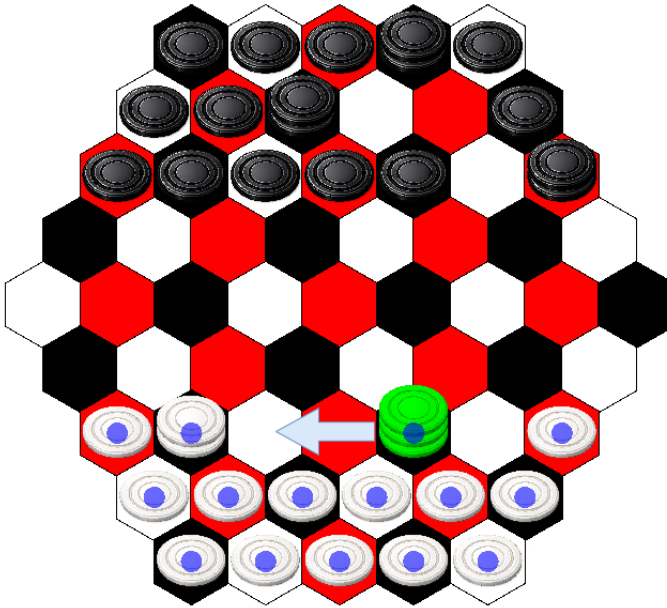


Figure 3.3: Example of the action of Player 1 moving 3 pieces from a stack to 2 tiles to the left

Generating all the possible actions for a board state is a straightforward task. It only requires to iterate over all the owned board tiles by the player who's turn is and, depending on the number of pieces on the stack and the surrounding pieces in the 6 directions of the

tile, check what actions are possible following the rules of the game described in a previous chapter. There isn't any major optimization that can be brought to this implementation of the actions generation.

For the minimization of memory usage of the **action** structure, it is sufficient to have the insight that to perfectly describe an action it is only needed to know the **from** tile, the **to** tile and the number of pieces moved from the **from** tile to the **to** tile. The tiles are numbered from 0 to 60, so 8 bits are sufficient to represent the **from** and **to** tiles. A player can theoretically move at most 35 pieces at the same time, so 8 bits are also sufficient to represent the number of moved pieces.

Therefore, the minimal representation of an action has only a size of:

$$8 \text{ bits} + 8 \text{ bits} + 8 \text{ bits} = 1 \text{ byte} + 1 \text{ byte} + 1 \text{ byte} = 3 \text{ bytes} \quad (3.2)$$

, which is really small.

### 3.1.3 GAME

Once the Board State and Action structures are implemented, including their useful methods, simulating a game is very simple.

A normal game starts in the position shown in Figure 2.1, where Player 1 (White) has to make the first move. From this point, each player chooses an action on their turn until:

1. one of the players has no actions left, resulting in a win for the other player
2. none of the players captured at least one enemy piece in the last 100 turns, resulting in a draw.

## 3.2 BASIC AGENTS AND USER INTERFACE

### 3.2.1 RANDOM AGENT

The most basic and easy to implement "AI" agent is the Random Agent. As its name suggests, the Random Agent chooses every time a random action. It's useful to play and to explore the game using such an agent because:

- it is used to analyze the different metrics of the game (e.g. duration, branching factor etc.).
- it is used to test how good other AI agents are. If an AI agent has a 50% win rate against the Random Agent, it means it's very bad. However, if the AI agent wins 100% of the games, it means it's already decent.
- it already creates an opponent who is easy to play against, which gives humans the opportunity to learn basic game strategies.

### 3.2.2 HUMAN AGENT AND USER INTERFACE

The Human Agent is also a basic agent because the action chosen at every turn is the one decided by a human.

To allow human players to compete against other humans or AI agents, a user interface was developed in C++ using the SFML library [5]. SFML is a high-level graphics library that simplifies the implementation of simple 2D games such as Tricolor.

When launching the game, it is needed to specify the two types of agents that will play against each other. The first one is Player 1 (White) and the second one is Player 2 (Black). Then, a window will pop on the screen (see Figure 3.4) and the game starts.

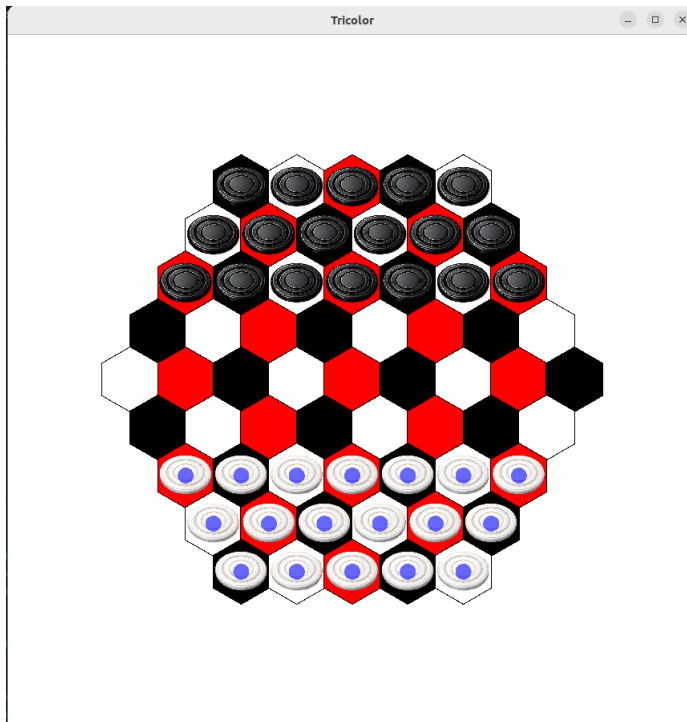


Figure 3.4: Window showing the beginning of a game of Tricolor



## 4

## GAME METRICS

## 4

This chapter presents some general metrics of Tricolor useful for better understanding the combinatorial complexity of the game. Some metrics are also a set of *Aesthetic Measurements*, which have the goal of measuring how *interesting* Tricolor is as a game. These concepts are defined in [6] and [7].

The origin and source of inspiration of these metrics can be found in the *Appendix A*.

In order to reproduce the results presented in this chapter, **1 million games** have been simulated between 2 **Random Agents**. This is a convenient approach because the moves made by the agents aren't biased by specific strategies or algorithms and the games are simulated fast.

In order to provide a visual representation of the computed metrics, for most of them, a *violin* plot is displayed (e.g. 4.1). It's a type of plot that shows the distribution of the y-values using surfaces colored in blue instead of individual dots. They are used when there are many data points that need to be represented on the graph for aesthetic reasons.

## 4.1 DURATION

### 4.1.1 METRIC

This metric measures the duration of a game in turns. A turn is equivalent to a move made by any of the 2 players. This metric is relevant for the performance of AI agents like *Alpha-Beta* and *MCTS* [4]. The Figure 4.1 shows the distribution of this metric measured in all the simulated games.

### 4.1.2 DATA INTERPRETATION

Looking at the plot, we can make the following observations:

- On average, a game lasts 173 turns. Comparing to chess, where a game lasts on average 80 turns [8]<sup>1</sup>, Tricolor's games depth is much bigger. This aspect has a

<sup>1</sup>In Shannon's paper about chess, a turn consists in two consecutive moves, one made by each player. However, in my measurements, a turn consists in only one move made by any player. Therefore, the number of turns as presented in Shannon's paper about chess was modified so that it satisfies my definition for the rest of this paper.

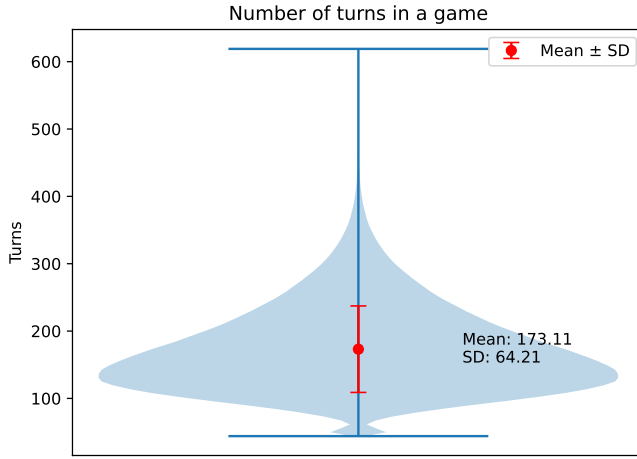


Figure 4.1: Plot with the number of turns in a game

huge impact on the performance of AI Agents, especially MCTS, which needs to perform a high amount of random game simulations in order to find the best move.

- The large standard deviation ( $\approx 64$  turns) indicates that game duration varies significantly depending on the Agents' behavior, suggesting that games do not follow a consistent flow.

## 4.2 BRANCHING FACTOR

### 4.2.1 METRIC

This metric measures how many possible actions any player can choose from, on average, during their turn. This metric is relevant for the performance of AI agents like *Alpha-Beta* and *MCTS* [4]. The Figure 4.2 shows the distribution of this metric measured in all the simulated games.

### 4.2.2 DATA INTERPRETATION

Looking at the plot, we can make the following observations:

- On average, a game has a branching factor of 47. Comparing to chess, where the average branching factor is 35 [8], Tricolor's branching factor is slightly higher. This aspect has a huge impact on the performance of AI Agents, especially for Alpha-Beta, as its time complexity heavily depends on the branching factor. The branching factor of the initial position of the game is 88 for both players (as it can be seen in Figure 2.1). As the game progresses, more and more pawns are being captured and imprisoned, which reduces the number of possible actions for both players during their turns. This explains why the average branching factor is 47.

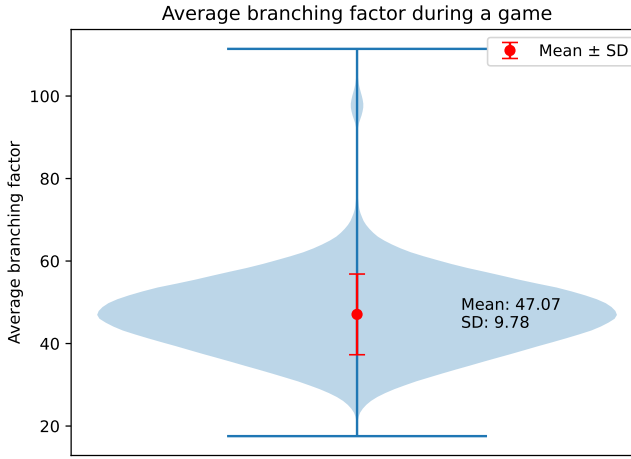


Figure 4.2: Plot with average branching factor in a game

- The standard deviation of  $\approx 10$  can be explained by the standard deviation obtained previously for the games duration metric. The shorter a game is, the smaller the average branching factor is, because pieces are captured and imprisoned very early in the game. The opposite is true for longer games.

## 4.3 GAME COMPLEXITY

### 4.3.1 GAME TREE COMPLEXITY

#### METRIC

This metric measures how many different games can be played starting from the initial position until the game ends. Two games are different if they differ by at least one action at any point. This metric is relevant for the performance of AI agents like *Alpha-Beta* and *MCTS* [4].

Computing the exact value of this metric for Tricolor is very complex. However, a good approximation is given by the following formula [4]:

$$GTC \approx b^d$$

, where:

- $GTC$  is the game tree complexity
- $b$  is the average game branching factor
- $d$  is the average game depth

Using the previously computed empirical values for the two game metrics in the formula, we obtain:

$$GTC \approx b^d = 47^{173} \approx 47^{1.2 \times 144} = (47^{1.2})^{144} \approx 100^{144}$$

This value is astronomical. As an example, the  $GTC$  of chess is  $\approx 10^{120} = 100^{60}$  [8], so Tricolor's  $GTC$  is much larger.

### 4.3.2 STATE TREE COMPLEXITY

#### METRIC

This metric measures how many different positions are achievable in Tricolor. In other words, it counts all the possible ways pieces can be placed on the board, including who owns each tile and which player's turn it is. This metric is relevant for the performance of AI agents like *Alpha-Beta* and *MCTS* [4].

In order to approximate the  $STC$  (State Tree Complexity), an upper bound is computed as it's too difficult to determine the  $STC$  using a simple mathematical formula. For this, the 4 following assumptions are made so that the upper bound is as small as possible:

1. Each player can have at most 18 pieces and at least 0 pieces on the board <sup>2</sup>.
2. Each piece, no matter its color, can be placed on any of the 61 tiles.
3. For each tile containing at least one piece of each color, 2 separate states must be considered <sup>3</sup>. However, counting only such tiles is too complex. In order to provide an upper bound, it is assumed each position has 18 pairs of white and black pieces on 18 different tiles. This results in  $2^{18}$  different states to multiply with each previous state.
4. Finally, a state also encodes the player's turn. Therefore, the final number of previous states must be multiplied by 2.

The trivial formula of combinations with repetitions <sup>4</sup> is used for the number of ways pieces can be placed on the 61 tiles. The following formula is obtained:

$$\begin{aligned}
 \text{UB}(STC) &= \underbrace{\sum_{w=0}^{w=18} \sum_{b=0}^{b=18} \binom{60+w}{w} \binom{60+b}{b}}_{\text{1st and 2nd assumptions}} \cdot \underbrace{2^{18}}_{\text{3rd assumption}} \cdot \underbrace{2}_{\text{4th assumption}} = \\
 &= \sum_{w=0}^{w=18} \binom{60+w}{w} \cdot \sum_{b=0}^{b=18} \binom{60+b}{b} \cdot 2^{19} = \binom{60+18+1}{18} \cdot \binom{60+18+1}{18} \cdot 2^{19} = \\
 &= \left( \frac{79}{18} \right)^2 \cdot 2^{19} \approx (2 \cdot 10^{15})^2 \cdot 2^{19} = 10^{30} \cdot 2^{21} \approx 10^{30} \cdot 2^{3 \cdot 3 \cdot 6.4} = 10^{30} \cdot (2^{3.3})^{6.4} \approx 10^{36}
 \end{aligned}$$

<sup>2</sup>A player can have 0 pieces if the last move consisted in a capture by the opponent.

<sup>3</sup>One for each player owning the tile

<sup>4</sup>Also called *stars and bars method*.

(4.1)

Therefore, an upper bound of the *STC* is  $10^{36}$ . It is notably high, but not larger than the *STC* of chess, which is  $\approx 10^{47}$  [8].

## 4.4 GAME OUTCOME

### 4.4.1 METRIC

This metric measures the percentage of games won by Player 1 (White), Player 2 (Black) and draws. The initial board setup is symmetrical, meaning no player has a positional advantage. It's useful to analyze from a *game aesthetics* [6] perspective whether making the first move brings or not an advantage in Tricolor. The Figure 4.3 shows the distribution of this metric measured in all the simulated games.

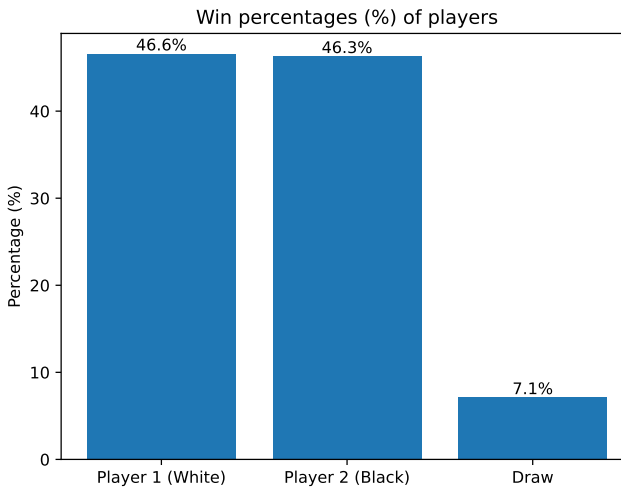


Figure 4.3: Plot with the win percentage of each player and draws

### 4.4.2 DATA INTERPRETATION

Looking at the plot, we can make the following observations:

- Player 1 (White) has a win percentage of 46.6% and Player 2 (Black) has a win percentage of 46.3%, which creates a difference of 0.3%. Given this very small difference in win percentage, we can confidently conclude that there is no obvious starting advantage for any of the players in Tricolor.
- The draw rate is  $\approx 7\%$ , which is notably low. It indicates that even with random moves, one of the players manages to capture or imprison all of the opponent's pieces.

## 4.5 STATE EVALUATION

All the metrics presented in the following section have the purpose of analyzing Tricolor from a *game aesthetics* perspective [6] and [7].

For the following metrics, a position evaluation of the game states is needed. The goal of the evaluation is to assess whether a certain position looks equal for both players, or if one of the players has an advantage and is more likely to win from there.

For this, each position is assessed with a score taking values in the range  $[-1, 1]$ , where a score of 0 represents an equal position, a score  $> 0$  represents an advantage for Player 1 (White) and a score  $< 0$  an advantage for Player 2 (Black). Computing the score follows this formula:

$$\frac{w_s - b_s}{w_s + b_s}, \in [-1, 1]$$

The total power of the black player is subtracted from the total power of the white player and then is divided by the sum of the two (in order to normalize the score to the range  $[-1, 1]$ ). We have:

$$w_s = \sum_{i=0}^{i=60} c_i * w_i$$

, where  $c_i$  is the number associated to the color of tile  $i$  (see rules 2) and  $w_i$  is the number of controlled white pieces on tile  $i$ <sup>5</sup>. Similarly,  $b_s$  is defined by considering the black pieces instead of the white pieces. In other words, the total power of a player is defined as the sum of the powers of all his owned stacks. The power of an owned stack is the product between the number of friendly pieces in the stack and the value associated to the color of the tile the stack sits on (see rules 2.3).

This position evaluation formula is relevant because:

- having more active pieces on the board results in higher chances of winning.
- controlling tiles with stronger colors results in more attack power and defense.

### 4.5.1 NARROWNESS

#### METRIC

This metric measures the average narrowness of all the turns of a game. The narrowness of a turn represents the number of *considerable* actions a player can choose from during that turn. An action is *considerable* if it improves or maintains the current score of the position for the player to move. The Figure 4.4 shows the distribution of this metric measured in all the simulated games.

#### DATA INTERPRETATION

Looking at the plot, we can make the following observations:

- The average narrowness is  $\approx 25.13$  moves with a standard deviation of  $\approx 5$ . Considering that the average branching factor is  $\approx 47$ , it means that in average about half the moves a player can choose from are worth looking at, which is a lot. This proves again that Tricolor is a complex combinatorial game.

<sup>5</sup>Imprisoned pieces are not counted.

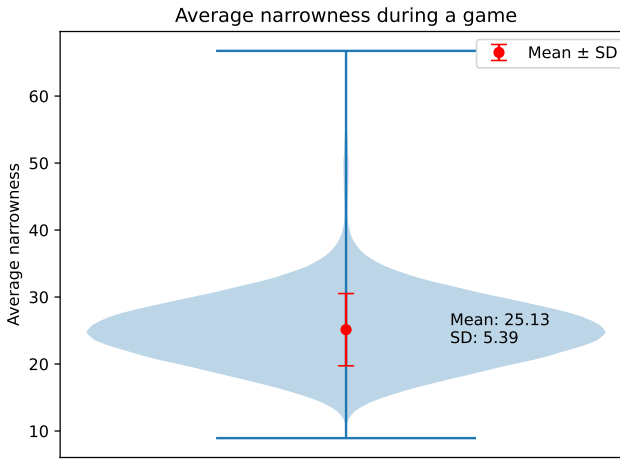


Figure 4.4: Plot with the average narrowness during a game

- When using a strong AI Agent with a good heuristic for Tricolor (see later 5), the position evaluation is much more complex than the one used in this chapter. The purpose of this metric is mainly to show how many actions a player can consider *playable* rather than truly good moves.

### 4.5.2 DECISIVENESS

#### METRIC

This metric measures the proportion of the game completed before the eventual winner establishes a decisive advantage that is maintained until the end of the game. This metric is only computed when a game has a winning player, so draws are discarded. The Figure 4.5 shows the distribution of this metric measured in all the simulated games.

#### DATA INTERPRETATION

Looking at the plot, we can make the following observations:

- The average decisiveness is 98.82 with a standard deviation of 6.91. This result suggests that most games don't have an obvious winner until the end.
- For Random Agents players, such a high decisiveness implies that Tricolor has an *uncertain* nature [6]. A game has an *uncertain* nature if the outcome of each game remains uncertain until the end, as long as both players maintain a vested interest in it, which is a good *game aesthetic*.

### 4.5.3 STABILITY

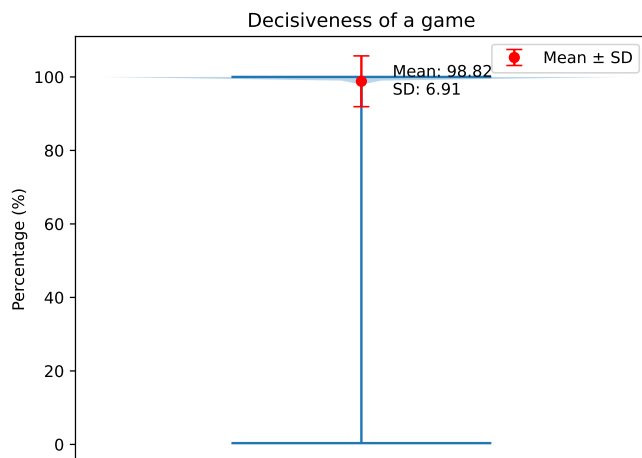


Figure 4.5: Plot with the game decisiveness

### METRIC

This metric measures the percentage of moves in a game that changed the leader. Its purpose is to assess how stable games are. The Figure 4.6 shows the distribution of this metric measured in all the simulated games.

### DATA INTERPRETATION

Looking at the plot, we can make the following observations:

- The average number of lead changing turns is  $\approx 16\%$  with a standard deviation of 6.79. As expected from the previous metrics, the stability of Tricolor games is low. This means that the leader tends to change frequently. It also has to do with Tricolor having an **uncertain** nature, as seen in the previous metric.

## 4.5.4 DRAMA

### METRIC

This metric measures the average change in position evaluation scores throughout a game. The purpose is to see how drastic are the changes in consecutive positions during a game. The Figure 4.7 shows the distribution of this metric measured in all the simulated games.

### DATA INTERPRETATION

Looking at the plot, we can make the following observations:

- The average drama is 0.07 with a standard deviation of 0.02. The score of a position evaluation is in the range  $[-1, 1]$ , so an average change of 0.07 is small.

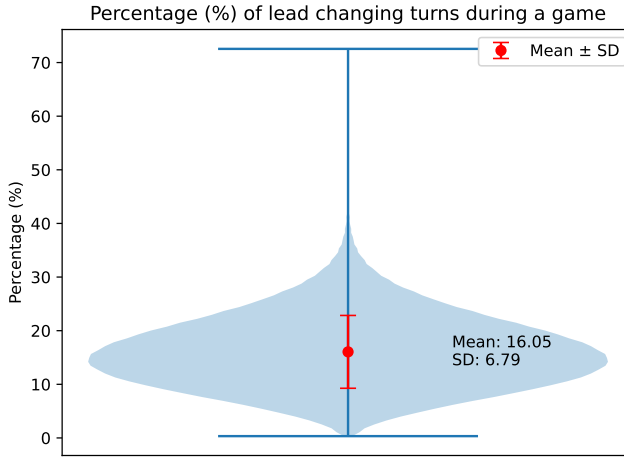


Figure 4.6: Plot with the percentage of lead changing turns during a game

- Even though the average game *drama* is small, a player in a disastrous losing position can manage to eventually win the game. This can happen when the losing player has many imprisoned pieces in an opponent stack but later regains their control. Therefore, Tricolor games have certain positions with high *drama*, which is a good game feature [6] and [7].

#### 4.5.5 GAME EVALUATIONS EXAMPLES

In this section we review and analyze two different games between Random Agents. We expect to find features that confirm the metrics found in the *State Evaluation* 4.5 section.

Each graph shows the score of each position of the game for every turn. The yellow line represents equal positions (score  $\approx 0$ ).

##### GAME 1

In Game 1 (see Figure 4.8), Player 1 (White) won.

Comments:

- In the first 60 turns, the drama is very low. However, even though Black gets a very good position at turn 69, White manages to regain the advantage and win the game.
- At around turn 76, White takes the lead and maintains it until winning the game.

##### GAME 2

In Game 2 (see Figure 4.9), Player 2 (Black) won.

Comments:

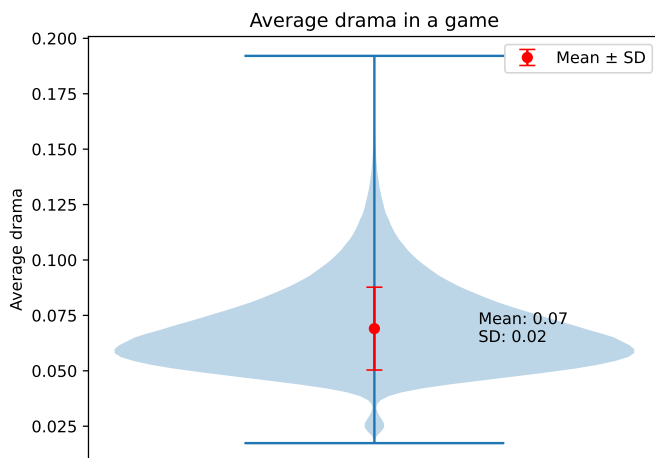
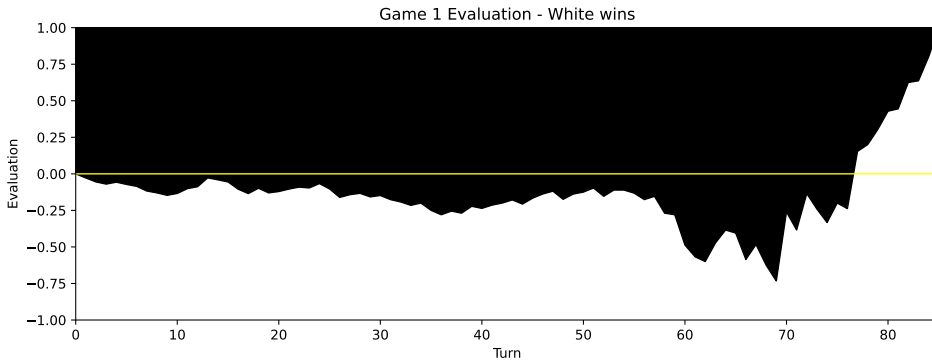


Figure 4.7: Plot with the average drama in a game

- Throughout the game, many lead changing moves happen. This is because the yellow line intersects many times the graph of the score function.
- The drama starts increasing significantly at turn 140.
- At around turn 170, Black takes the lead and maintains it until winning the game, which happens very late in the game.



4

Figure 4.8: Plot with game 1 evaluation

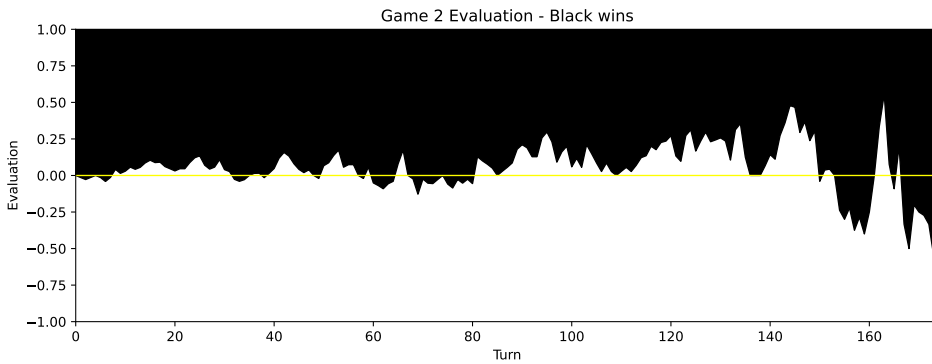


Figure 4.9: Plot with game 2 evaluation



## 5

# GAME ANALYSIS FOR ADVANCED AI AGENTS

5

So far, the only autonomous agent we have seen and used for computing the metrics of Tricolor is the Random Agent. In this chapter, we will analyze some mathematical properties of Tricolor in order to have an overall picture of the AI agents' architecture best suited for the game. Then, we will introduce three advanced agents with the goal of analyzing their performance and decide which one is the best.

## 5.1 GAME COMPLEXITY

Defining a simple set of rules in order to determine what move to play at each turn isn't easy for Tricolor because it's a complex combinatorial game. Therefore, any good AI agents for this game will need to explore in some measure the state tree of the game in order to determine the best move <sup>1</sup>. This means the performance of the agents is highly dependent on the average game depth and the average branching factor, as these parameters influence the size of the game state tree.

In the previous chapter 4 where we analyzed the different metrics of Tricolor, we have seen that the average game depth is  $\approx 173$  turns and the average branching factor is  $\approx 47$ . We have also seen that the game tree complexity is  $\approx 100^{144}$  and the state tree complexity is  $\approx 10^{36}$ , which is a lot to explore. Therefore, we introduce game heuristics designed to provide fast and reliable evaluations of positions that cannot be explored further during the search due to time constraints.

## 5.2 HEURISTICS

### 5.2.1 FIRST INSIGHT

The goal of the game Tricolor is to make the opponent run out of moves during their turn. In order to achieve this, you need to capture or imprison all of the opponent's pieces <sup>2</sup>.

<sup>1</sup>Ideally, the entire state tree of the game is explored.

<sup>2</sup>In theory, a player can run out of legal moves even if he still controls one or more tiles. This happens when the losing player only has stacks fully surrounded by stronger opponent stacks he can't capture. However, such

Therefore, a first solid heuristic is to compare the number of controlled pieces on the board between the players.

### 5.2.2 SECOND INSIGHT

Capturing and imprisoning opponent pieces but also defending your own requires your stacks to be on stronger tiles <sup>3</sup>. Therefore, a second heuristic is to compare the accumulated power of all the controlled stacks of pieces on the board between the players.

### 5.2.3 THIRD INSIGHT

Tricolor has a rule specifying that if a player can imprison or capture one or more enemy stacks during their turn, they are forced to play such a move. If we ignored this rule, players could always force any game to end in a draw by stacking at least 3 pieces on any red tile, which will never be captured. This is because the power of such a stack is  $3 \times 3 = 9$  and it's impossible to form a stack with power  $\geq 10$  in order to at least imprison it. However, because this rule exists, players can make their opponent "break" the power of their stacks placed on red tiles by forcing them to capture nearby pieces. This leads to a game strategy called "Single piece baiting".

To better understand it, let's consider the scenario in Figure 5.1:

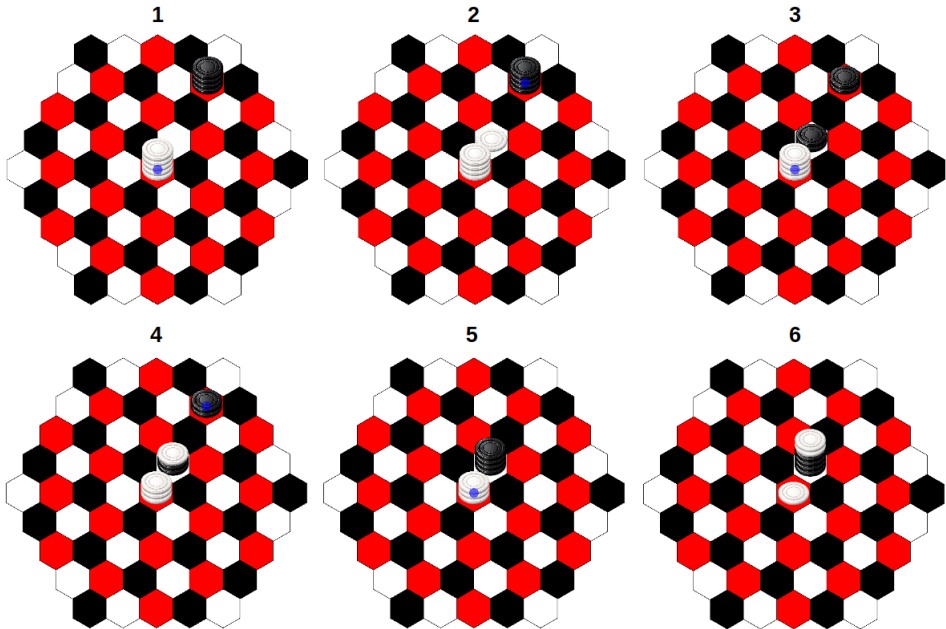


Figure 5.1: Figure displaying how the "single piece baiting" strategy used by the white player defeats the powerful stack of the black player.

strategies can be neglected when designing heuristics as surrounding weaker opponent stacks doesn't bring a greater advantage than capturing or imprisoning them.

<sup>3</sup>Red colored tiles triple the power, black doubles it and white has no effect.

In the scenario presented in Figure 5.1, the white player (which we will refer to as "White") has a stack of 5 pieces on the red central tile and the black player (which we will refer to as "Black") has a stack of 4 pieces on a red tile close to White. In the second image, White moves a white piece on a white tile, which forces the Black to capture it with at least 2 pieces (because the tile is at distance 2 of their stack). White then repeats the "single piece baiting" strategy until Black runs out of pieces. Finally, in the sixth image, White manages to imprison all of Black's pieces and wins the game <sup>4</sup>. The reason why this strategy works is because the "victim" player (in this case Black) is further away from the contested tile the attacker (in this case White) initially moved its piece to. Now, the question is, can a stack with less pieces win against a stack with more pieces using a "single piece baiting" attack? The answer is **no**.

**Proof:**

First, we are only interested in positions where both stacks are on red tiles. This is because during a game, good players will try to move their pieces on the red tiles as fast as possible. Also, we are only interested in positions where the tile used as bait has the color white. This is because using a black or red tile as bait will give the defender the advantage of having a better defended stack, which will force the attacker to use even more pieces to continue the attack. By the geometry of the board, which has a hexagonal tiling, we can observe that for any tile, all the tiles at distance 3 have the same color (see Figure 5.2 as an example). This means a stack on a red tile can't be attacked with a "single piece baiting" attack using a bait tile at distance 3 of the defender. In the best case, such an attack will happen when using a white bait tile at distance two (e.g. in Figure 5.1 where the white bait tile is at distance two of the black stack).

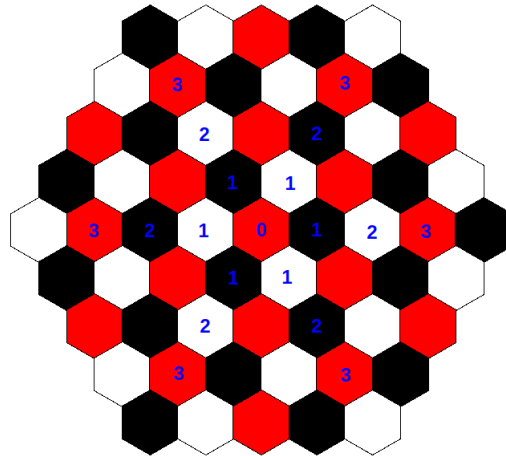


Figure 5.2: Figure showing that all the tiles at distance 3 of the central red tile are also red.

Now that we established the scenarios we are interested in for the "single piece bait"

<sup>4</sup>Notice that White still has one piece left on the initial central tile, which means this strategy also works when both players have stacks of the same size.

attack, let's show that if the attacker has less pieces than the defender, the attack will never succeed. The following flowchart 5.3 proves it by enumerating all the possible moves the attacker has.

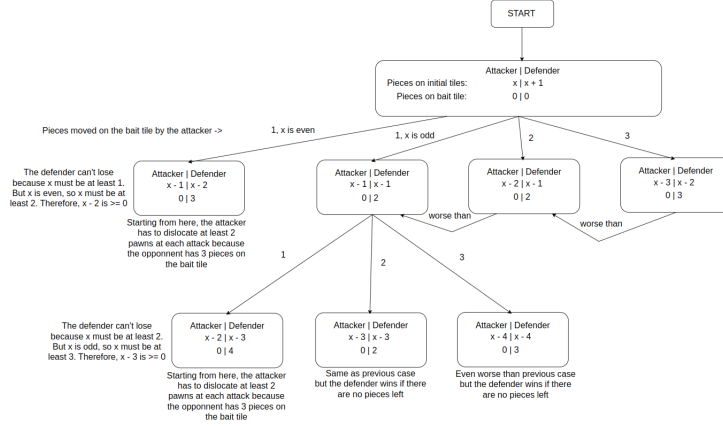


Figure 5.3: Flowchart of all possible move sequences during a "single pawn bait" attack.

Finally, a third heuristic is to prioritize stacking pieces on a minimal number of tiles. This is because having a smaller number of big stacks is better than having a bigger number of small stacks. As we just proved earlier, a big stack on a red tile won't be the victim of "single piece bait" attack of a smaller stack, but the opposite is true.

## 5.2.4 COMBINING THE HEURISTICS

Choosing how to combine these three heuristics is very important because it defines their priority. Analyzing multiple games with different combinations of the two heuristics led to the conclusion that the priorities should be established in the following way:

- Comparison of the **controlled number of pieces** on the board between the players. (Priority 1)
- Comparison of the **accumulated power of all the controlled stacks of pieces** on the board between the players, where the power of a stack is:

$$\text{number of pieces}^2 * \text{color}$$

with the goal of **preferring taller stacks**. (Priority 2).

This choice of priorities is the consequence of the fact that, when mixing the 3 heuristics equally, an AI agent will rather choose to sacrifice pieces only to place other stacks on stronger tiles. This is a bad strategy because it only improves the position in the short term.

## 5.3 ALPHA-BETA AGENT

The **Alpha-Beta Agent** uses the well-known alpha-beta pruning algorithm [4] to determine the best move at each turn in one second. This algorithm is an optimization of Minimax by pruning branches not worthy visiting during the search. After the maximum depth is reached and the obtained position is not the end of the game, the heuristic built previously 5.2 is used to evaluate and score the position.

In addition to the basic alpha-beta algorithm, two improvements are considered:

- **Iterative Deepening** – a technique that tries all the possible search depths starting from 1, as long as the available time for the player's turn is not over. The improvement is that, instead of having a fixed depth for each turn, the highest possible depth is chosen depending on the current game position. Even if repeating the algorithm for each tested depth wastes time, it is better that fixing a smaller depth for all the played turns.
- **Sorting Actions by Evaluation Score** – before iterating over the possible actions of a player in a certain position during the search, the list of actions is sorted in decreasing order. This maximizes the chances of pruning future branches earlier in exchange of a small overhead for sorting the lists of actions.

5

## 5.4 MONTE CARLO TREE SEARCH AGENT

The **Monte Carlo Tree Search Agent (MCTS)** uses the well-known MCTS algorithm to determine the best move at each turn in one second [4]. In essence, it consists in running many game simulations starting from the current position and evaluate how good each move is based on the result of each simulation. The algorithm is divided into the four classical steps, with some particularities:

- **Selection** – the policy used for choosing a child at each fully expanded level of the tree is the **Upper Confidence bounds applied to Trees (UCT)**. The selected child is the one maximizing:

$$\frac{w_i}{n_i} + c \cdot \sqrt{\frac{\ln N}{n_i}}$$

, where

- $w_i$  is the number of wins for child  $i$
- $n_i$  is the number of times child  $i$  was visited
- $N$  is the total visits of the parent
- $c$  is the exploration constant. In our implementation, the value of  $c$  is chosen to be  $\sqrt{2}$ , as it was empirically proved to have the best results on average [9].
- **Expansion** – only one child is expanded at each iteration.
- **Simulation** – for a game resulting in a win, the score 1 is assigned and 0 otherwise. Draws are not handled.
- **Backpropagation** – classical implementation.

## 5.5 EXPERT ITERATION

In progress

# 6

## PERFORMANCE ANALYSIS OF THE AGENTS

In this chapter we analyze the performance of each agent and determine which one is the best. For this, we simulated 100 games between each pair of distinct agents <sup>1</sup>. Each agent played 50 games as white and 50 games as black for fairness reasons.

At the end of this chapter we will also analyze the same game metrics as in chapter 4, but this time using the advanced agents instead of the random agents.

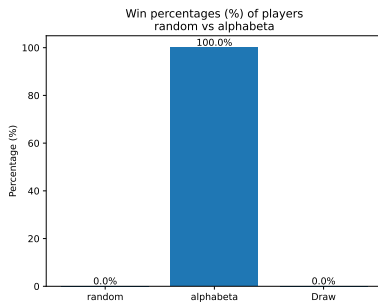
6

### 6.1 RANDOM

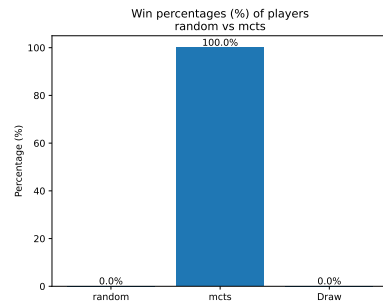
As expected, the Random Agent loses all the games against the Alpha-Beta Agent and the MCTS Agent (Figure 6.1). This already suggests that the advanced AI agent are correctly implemented.

---

<sup>1</sup>We only simulated 100 games because each advanced agent takes 1 second to make a move. Given the high average game depth 4.1, more than 100 games would take a lot of time to run.



(a) Random vs Alpha-Beta



(b) Random vs MCTS

Figure 6.1: Plots displaying the win percentage of the Random Agent vs the Alpha-Beta Agent and the MCTS Agent.

## 6.2 ALPHA-BETA VS MCTS

Looking at the plot (Figure 6.2), we can see that the Alpha-Beta won 93% of the games, whereas MCTS only won 2% of the games. This result clearly indicates that the Alpha-Beta agent is much better than the MCTS agent.

6

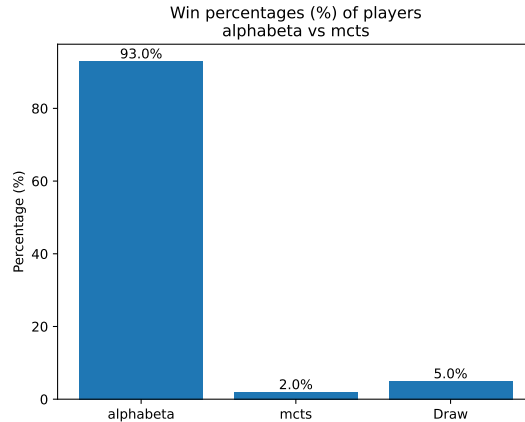


Figure 6.2: Plot displaying the win percentage of the Alpha-Beta Agent vs the MCTS Agent.

## 6.3 GAME METRICS USING AI AGENTS

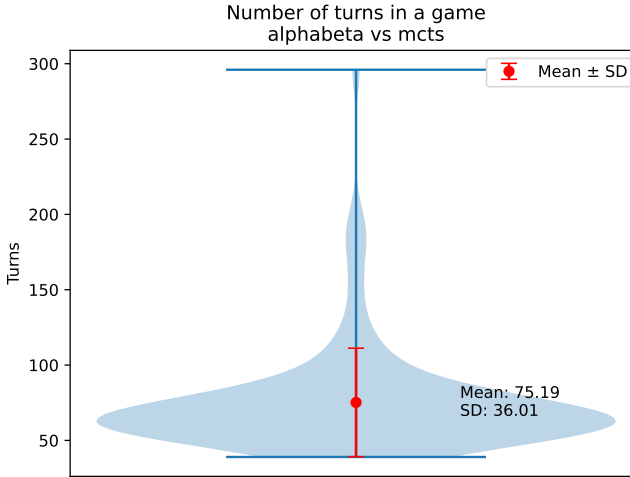
In this section, the metrics introduced in Chapter 4 are recomputed for games played between an Alpha-Beta agent and an MCTS agent. The objective is to compare these results with those obtained from simulations between two Random agents and to discuss the observed differences. Only the metrics that exhibit different behavior between the two

experimental setups are considered.

Additional metrics are in the Appendix A.

### 6.3.1 DURATION

The Figure 6.3 shows the distribution of this metric measured in all the simulated games.



6

Figure 6.3: Plot with the number of turns in a game

Looking at the plot, we can make the following observations:

- On average, a game lasts 75 turns. The average game depth between two Random Agents is 173 turns. This shows that the Alpha-Beta Agent manages to beat the MCTS Agent very fast, which reduces the games durations quite drastically.

### 6.3.2 BRANCHING FACTOR

The Figure 6.4 shows the distribution of this metric measured in all the simulated games.

Looking at the plot, we can make the following observations:

- On average, a game has a branching factor of 75. The average branching factor between two Random Agents is 47. This large difference can be justified by the fact the the Alpha-Beta Agent spends its first 15-20 turns by stacking more and more pieces on red tiles. Once it's done, it manages to beat the MCTS Agent fast. Therefore, very few turns are spent with a little number of pieces <sup>2</sup> on the board during the game.

### 6.3.3 DECISIVENESS

The Figure 6.5 shows the distribution of this metric measured in all the simulated games.

<sup>2</sup>and therefore a small branching factor

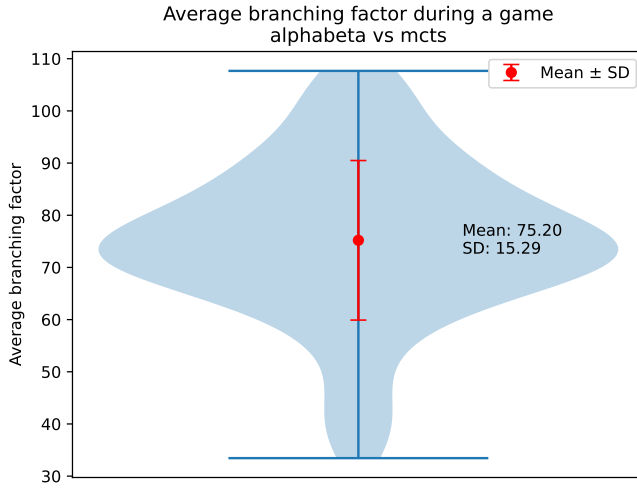


Figure 6.4: Plot with average branching factor in a game

## 6

Looking at the plot, we can make the following observations:

- Waiting for result (I made a mistake in the metric)

### 6.3.4 STABILITY

The Figure 6.6 shows the distribution of this metric measured in all the simulated games.

Looking at the plot, we can make the following observations:

- The average number of lead changing turns is  $\approx 4\%$ . The average number of lead changing turns between two Random Agents is  $\approx 16\%$ . This difference is justified by the fact that the Alpha-Beta Agent is much better than the MCTS Agent. Alpha-Beta tends to always have a better position and therefore the lead change frequency is very low.

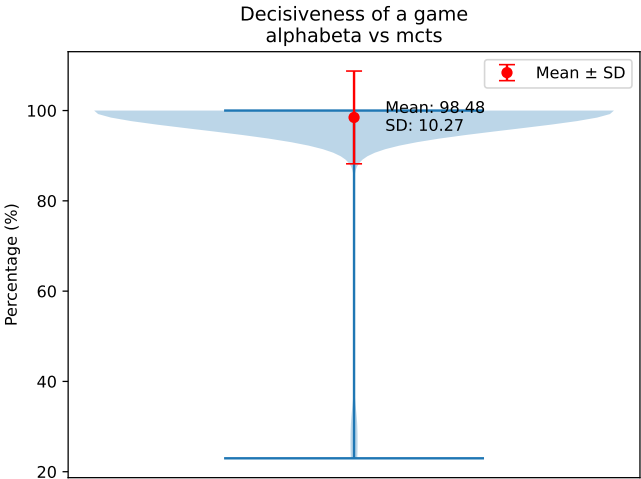


Figure 6.5: Plot with the game decisiveness

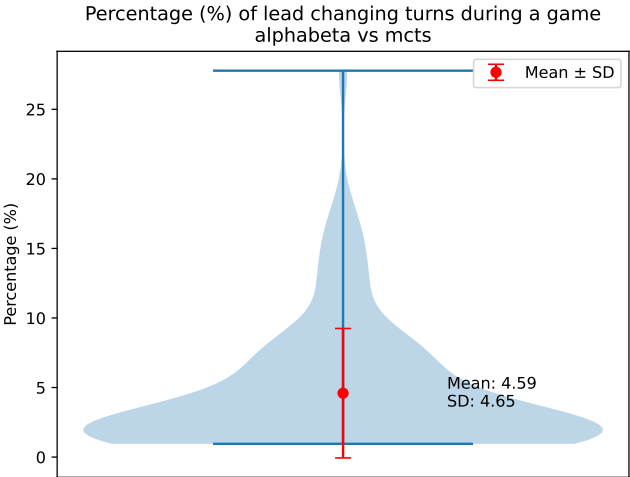


Figure 6.6: Plot with the percentage of lead changing turns during a game





## GAME METRICS PLOTS

This chapter of the appendix contains all the metrics plots computed for Tricolor, in addition to the ones presented in the chapter *Game Metrics*[4].

These are the following metrics, as well as their definition and origin of inspiration:

- **Duration** – measures the duration of a game in turns. A turn is equivalent to a move made by any of the 2 players.  
Origin: inspired from [6] and [8].
- **Branching Factor** – measures how many possible actions any player can choose from, on average, during their turn.  
Origin: inspired from [8].
- **Decision Moves** – measures what percentage of the moves made by any player during a game aren't forced. A move isn't forced if the player can choose between at least 2 actions during their turn.  
Origin: inspired from [10].
- **State Repetition** – measures how many times the same states are repeated during a game. A state is repeated whenever the Tricolor board reaches a position equivalent to a previous one during the same game. Two positions are equivalent if they contain the same pieces on the same tiles, controlled by the same players, and with the same player to move.  
Origin: inspired from the [10].
- **Decision Factor** – measures how many possible actions any player can choose from, on average, during their turn, ignoring turns where only one move is possible.  
Origin: inspired from [10].
- **Board Sites Occupied** – measures how many board tiles were occupied by any player at least once during the game. Initially, both players occupy 18 tiles each, so

## A

36 tiles in total. The board has a total of 61 tiles, so, for each game, the value of this metric is between 36 and 61.

Origin: inspired from [10].

- **Move Distance** – measures the average distance traveled by pieces when they move around the board during a game. In Tricolor, a stack of pieces can move at a distance of 1, 2 or 3 tiles, depending on the number of pieces on the stack (see Game Rules 2).

Origin: inspired from [10].

- **Piece Number** – measures the average number of pieces on the board during a game. Players start with 18 pieces each, so there are 36 pieces in total initially on the board.

Origin: inspired from [10].

- **Game Outcome** – measures the percentage of games won by Player 1 (White), Player 2 (Black) and draws.

Origin: inspired from [6].

- **Narrowness** – measures the average narrowness of all the turns of a game. The narrowness of a turn represents the number of considerable actions a player can choose from during that turn. An action is considerable if it improves or maintains the current score of the position for the player to move.

Origin: inspired from [7] and [10].

- **Decisiveness** – measures the proportion of the game completed before the eventual winner establishes a decisive advantage that is maintained until the end of the game. This metric is only computed when a game has a winning player, so draws are discarded.

Origin: inspired from [7].

- **Lead Change** – measures the average number of possible actions during a turn that change the leader of the game. The leader of the game is the player who has a positional advantage.

Origin: inspired from [10].

- **Stability** – measures the percentage of moves in a game that changed the leader. Its purpose is to assess how stable games are.

Origin: inspired from [6] (called *Uncertainty* in the paper) and [10].

- **Drama** – measures the average change in position evaluation scores throughout a game. Its purpose is to see how big the jumps are on average between consecutive turns during a game in terms of position evaluation.

Origin: inspired from [6], [7] and [10].

## A.1 RANDOM AGENT VS RANDOM AGENT SIMULATIONS

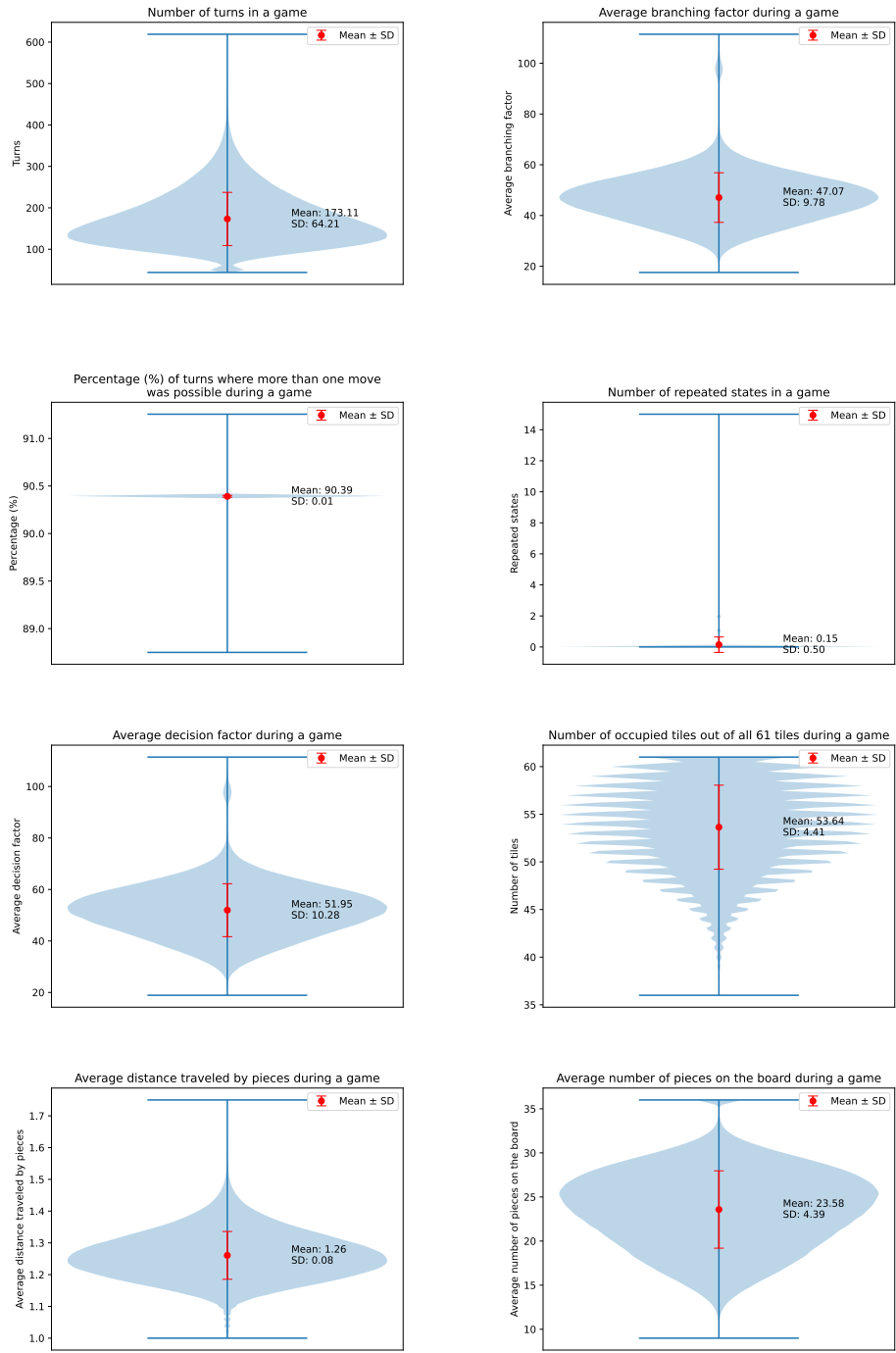


Figure A.1

A

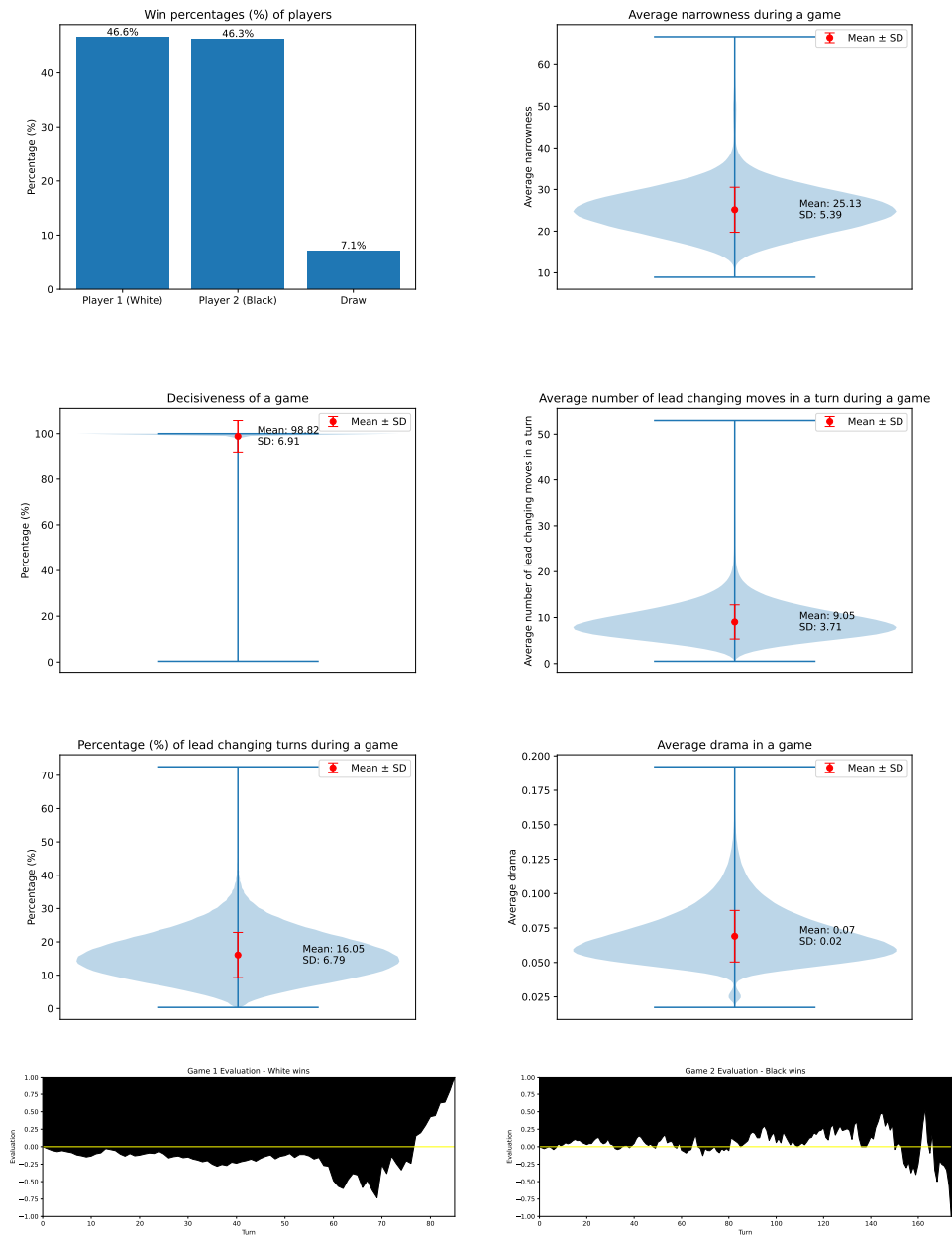


Figure A.2

## A.2 ALPHA-BETA AGENT VS MCTS AGENT SIMULATIONS

**A**

A

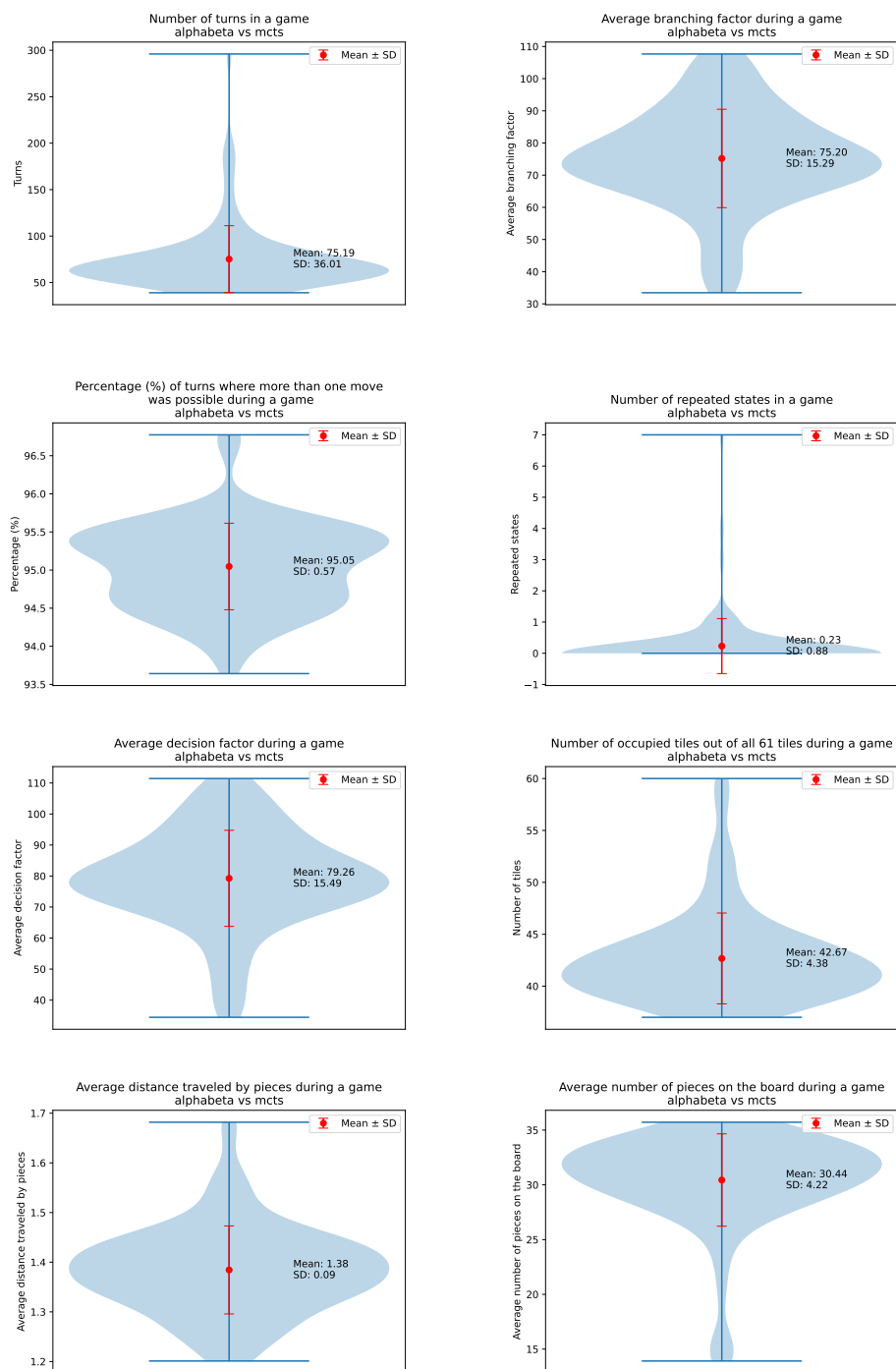


Figure A.3

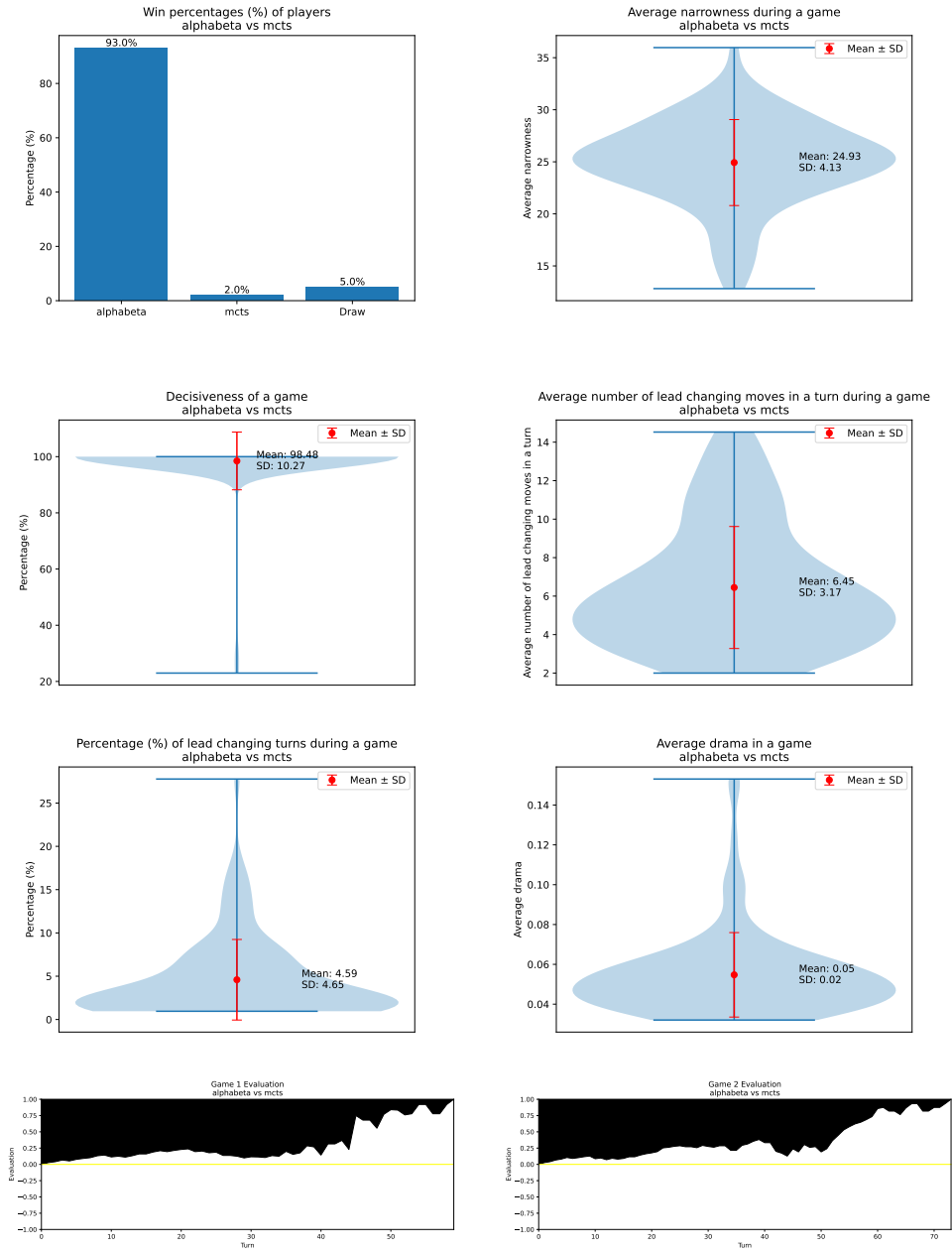


Figure A.4



# BIBLIOGRAPHY

URLs in this thesis have been archived on Archive.org. Their link target in digital editions refers to this timestamped version.

## REFERENCES

- [1] Rougetet, Lisa and Hirth, Tiago, Maurice Kraitchik (1882–1957) and his contributions to recreational mathematics. Available at SSRN: <https://ssrn.com/abstract=6074494> or <http://dx.doi.org/10.2139/ssrn.6074494>.
- [2] Tiago Hirth and Lisa Rougetet. Mathematical games from the sphinx 1931–1939. Unpublished paper. Supported by COST Action CA22145 – GameTable | Computational Techniques for Tabletop Games Heritage. <https://gametable.network/>.
- [3] Soemers, Dennis JNJ, Vegard Mella, Cameron Browne, and Olivier Teytaud. "Deep learning for general game playing with Ludii and Polygames." ICGA Journal 43, no. 3 (2022): 146-161.
- [4] Russell, Stuart J. Artificial intelligence a modern approach. Pearson Education, Inc., 2010.
- [5] Laurent Gomila. Simple and fast multimedia library (sfml). <https://www.sfml-dev.org>, 2025.
- [6] Browne, Cameron, and Frederic Maire. "Evolutionary game design." IEEE Transactions on Computational Intelligence and AI in Games 2, no. 1 (2010): 1-16.
- [7] Browne, Cameron Bolitho. "Automatic generation and evaluation of recombination games." PhD diss., Queensland University of Technology, 2008.
- [8] Shannon, Claude E. "XXII. Programming a computer for playing chess." The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science 41, no. 314 (1950): 256-275.
- [9] Auer, Peter, Nicolo Cesa-Bianchi, and Paul Fischer. "Finite-time analysis of the multi-armed bandit problem." Machine learning 47, no. 2 (2002): 235-256.
- [10] Ludii Portal, Concepts <https://ludii.games/searchConcepts.php>.

